

Strategy Guide: Integrating CWE Top 25 into Your Vulnerability Scanner

1. Why use the CWE Database?

Instead of creating custom names for vulnerabilities (e.g., "Bad Input Error"), using CWE (Common Weakness Enumeration) IDs allows your scanner to:

1. **Standardize Output:** Security teams use tools like Jira, DefectDojo, or SonarQube. These tools rely on CWE IDs to categorize issues.
2. **Enrich Data:** You don't need to hardcode descriptions in your scanner code. You simply detect the issue, tag it with an ID (e.g., CWE-89), and pull the description/remediation from your JSON database.
3. **Prioritize:** The score and rank fields in the JSON help you tell the user which bugs to fix first.

2. Architecture for Mobile & Web Scanning

To support both Mobile and Web using the JSON data provided, structure your scanner as follows:

A. The Detection Engine (The "Brain")

Your scanner needs specific "Modules" or "Rules" that map to the CWEs in the JSON.

For Web (URL) Scanning:

- **Input Injection Module:** Maps to CWE-79 (XSS) and CWE-89 (SQLi).
 - *Action:* Fuzz parameters with ' or <script>.
- **Config Module:** Maps to CWE-319 (Cleartext Transmission).
 - *Action:* Check if the site serves over HTTP or has expired SSL.
- **Auth Module:** Maps to CWE-287 (Improper Auth).
 - *Action:* Check for default admin pages or weak password policies.

For Mobile (APK/IPA) Scanning:

- **Manifest Analysis:** Maps to CWE-276 (Incorrect Default Permissions).
 - *Action:* Parse AndroidManifest.xml for android:exported="true".
- **Binary Analysis (Strings):** Maps to CWE-798 (Hardcoded Credentials).
 - *Action:* Regex search the decompiled code for api_key, access_token.
- **Insecure Storage:** Maps to CWE-312 (Cleartext Storage - typically linked to CWE-200).
 - *Action:* Check SharedPreferences or .plist files for sensitive data.

3. How to use the JSON Fields

When your scanner detects an issue, look up the CWE ID in the JSON file to populate your report.

JSON Field	Usage in Scanner
platforms	Routing Logic. If the platform is "Mobile (Native)", do not run this check against a Web URL. Use this to filter which rules to load.
detection_logic	Developer Hint. This field in the JSON helps <i>you</i> (the scanner developer) write the rule. It is not necessarily for the end user.
mitigation	Report Content. Display this prominently in the final report so the user knows how to fix the bug.
score	Sorting. Sort your final report vulnerabilities by this score (Descending) to show the most dangerous issues first.

4. Keeping Data Fresh

The CWE Top 25 changes every year.

1. **Manual Update:** Download the new list from MITRE annually and update your `cwe_top_25_vulnerabilities.json`.
2. **Automated:** Write a script to scrape `cwe.mitre.org` and regenerate the JSON structure.

5. Next Steps

1. **Load the JSON** at the start of your scanner execution.
2. **Filter Rules:** If the user selects "Scan Mobile", filter your internal rules to only those where platforms includes "Mobile".
3. **Scan & Match:** When a rule triggers, grab the metadata from the JSON object using the ID.
4. **Export:** Produce the final report in JSON/PDF using the rich descriptions provided.